Informe de la Pràctica 1: Blink i Activitats Complementàries

1. Introducció

1.1 Objectius

L'objectiu principal de la pràctica és produir el parpelleig periòdic d'un LED i utilitzar la sortida sèrie per depurar el programa. A partir d'aquí, es demanen les següents fites:

- Modificar el codi perquè enviï de manera sincronitzada els missatges "ON" i "OFF" al port sèrie.
- Pujar el codi desenvolupat a un repositori de GitHub.
- Llegir un valor d'un convertidor A/D, treure'l pel port sèrie i replicar-lo per un pin D/A.
- Llegir el valor del sensor de temperatura intern i treure'l pel port sèrie.
- Avaluar i justificar quin és el temps lliure del processador en aquest programa.
- Activitat complementària: Entendre la gestió de llibreries a PlatformIO, llegir característiques del maquinari i controlar el LED RGB integrat mostrant els 6 colors bàsics i el blanc.

1.2 Material Utilitzat

S'ha utilitzat una placa de desenvolupament basada en el microcontrolador **ESP32-S3** (models ESP32-S3-DevKitC-1 o YD-ESP32-S3). L'entorn de programació escollit és PlatformIO. A nivell físic, s'utilitzen els pins GPIO d'entrada/sortida digital, els pins analògics (ADC), els pins de transmissió sèrie (U0TXD i U0RXD) i el bus unifilar connectat al LED RGB integrat (tipus WS2812).

2. Desenvolupament i Implementació

2.1 Codi Base i Modificació per Enviar ON/OFF al Port Sèrie

Un cop desenvolupat, el programa s'ha versionat a GitHub utilitzant les comandes git add, git commit i git push com a requeriment del lliurament.

```
#include <Arduino.h>

const int ledPin = 2;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("ON");
  delay(500);

  digitalWrite(ledPin, LOW);
  Serial.println("OFF");
```

```
  delay(500);
}
```

Explicació del codi:

- **const int ledPin = 2:** Assignem el pin GPIO on està connectat el LED.
- **setup():** S'executa un sol cop a l'inici. pinMode configura el pin 2 com a sortida (OUTPUT) i Serial.begin(115200) inicialitza el port de comunicació sèrie a 115200 bauds per parlar amb el PC.
- **loop():** Bucle infinit. S'encén el LED amb digitalWrite(ledPin, HIGH), s'envia el text "ON" pel port sèrie, i s'espera 500 mil·lisegons amb delay(500). Tot seguit s'apaga posant-lo a LOW, s'imprimeix "OFF" i s'espera mig segon més, tancant el cicle periòdic.

2.2 Modificació per Conversió Analògica-Digital (ADC) i Emulació D/A

La família ESP32-S3 no disposa de maquinari DAC dedicat (Convertidor Digital-Analògic). Per complir la pràctica, llegim l'entrada analògica normalment i utilitzem un senyal modulat d'alta freqüència (Sigma-Delta o PWM) com a sortida per simular l'acció del D/A.

```
#include <Arduino.h>

const int pinADC = 10;
const int pinDAC_Sim = 11;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // Resolució de 12 bits (0-4095)
}

void loop() {
  int valorAnalogic = analogRead(pinADC);

  Serial.print("Valor llegit A/D: ");
  Serial.println(valorAnalogic);

  // Escalem de 12-bits a 8-bits per la sortida PWM
  int valorSortida = map(valorAnalogic, 0, 4095, 0, 255);
  analogWrite(pinDAC_Sim, valorSortida);

  delay(100);
}
```

Explicació del codi:

- **analogReadResolution(12):** Estableix la lectura ADC a 12 bits, de manera que els valors aniran de 0 a 4095.
- **analogRead(pinADC):** Llegeix el voltatge físic del pin 10. Aquest valor s'imprimeix al monitor sèrie.
- **map(...):** Redueix l'escala lineal de 0-4095 a 0-255 per poder injectar-ho com a cicle de treball (Duty Cycle).
- **analogWrite():** En absència de DAC en l'ESP32-S3, generem un senyal PWM pel pin 11.

2.3 Modificació del Sensor de Temperatura Intern

```
#include <Arduino.h>

#ifdef __cplusplus
extern "C" {
#endif
uint8_t temprature_sens_read();
#ifdef __cplusplus
}
#endif

void setup() {
  Serial.begin(115200);
}

void loop() {
  uint8_t rawTemp = temprature_sens_read();
  float tempCelsius = (rawTemp - 32) / 1.8;

  Serial.print("Temperatura CPU: ");
  Serial.print(tempCelsius);
  Serial.println(" °C");
  delay(5000);
}
```

Explicació del codi:

- Es fa servir el bloc extern "C" per poder importar i utilitzar la funció interna temprature_sens_read() directament del codi font originari en C.
- Un cop llegit el valor en brut (Fahrenheit), s'aplica una operació matemàtica senzilla per passar-lo a Celsius.
- Imprimeix el resultat. És important remarcar que aquest sensor mesura la temperatura *interna* del nucli operatiu, i no la temperatura ambient de l'habitació.

3. Temps lliure del processador en el programa

Per respondre a la qüestió del temps lliure demanada als exercicis , cal entendre l'entorn FreeRTOS de l'ESP32. A diferència de microcontroladors antics, la instrucció delay(500) no és un bucle d'espera activa (que malgastaria CPU). El sistema tradueix aquest delay a la funció vTaskDelay() del sistema operatiu en temps real.
Aquesta funció posa la tasca principal en un estat de "Bloqueig" (Blocked State). Com que la tasca del LED està bloquejada durant 500 ms i no n'hi ha d'altres, el processador entra directament a executar la **"Idle Task"** (Tasca inactiva).
Com que encendre un LED o emetre dades per l'UART triga només uns pocs microsegons de temps de rellotge i el microprocessador passa gairebé tot el segon restant "adormit" a causa del delay(), podem concloure que **el temps lliure efectiu del processador en aquest programa bàsic és de gairebé el 99,9%**.

4. Activitat Complementària (LED RGB)

4.1 Gestió de Llibreries a PlatformIO

L'Arduino IDE clàssic instal·la les llibreries globalment. Si s'actualitza una llibreria per a un projecte nou, podria trencar el codi dels projectes antics que utilitzaven versions anteriors. A PlatformIO les llibreries es descarreguen i gestionen de manera aïllada **per a cada projecte**

(configurades a platformio.ini). Això garanteix l'estabilitat i la independència de cada codi.

4.2 Codi Llum RGB i Lectura de Maquinari

S'utilitza la llibreria Adafruit_NeoPixel per controlar el LED RGB.

```
#include <Arduino.h>
#include <Adafruit_NeoPixel.h>

#define PIN       48
#define NUMPIXELS 1
Adafruit_NeoPixel pixels(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);

void setup() {
  pixels.begin();
  pixels.setBrightness(10); // Lluminositat mínima per seguretat

  Serial.begin(115200);
  delay(500);

  Serial.println("\nInfo Xip:");
  Serial.printf("Model: %s\n", ESP.getChipModel());
  Serial.printf("Cores: %d\n", ESP.getChipCores());

  String MACString = "";
  uint64_t chipid = ESP.getEfuseMac();
  for (int i=0; i<6; i++) {
    if (i > 0) MACString.concat(":");
    uint8_t Octet = chipid >> (i * 8);
    MACString.concat(String(Octet, HEX));
  }
  Serial.println("MAC: " + MACString);
}

void loop() {
  // Primaris
  pixels.setPixelColor(0, pixels.Color(255, 0, 0));
  pixels.show(); delay(1000); // Rojo

  pixels.setPixelColor(0, pixels.Color(0, 255, 0));
  pixels.show(); delay(1000); // Verde

  pixels.setPixelColor(0, pixels.Color(0, 0, 255));
  pixels.show(); delay(1000); // Azul

  // Secundaris
  pixels.setPixelColor(0, pixels.Color(0, 255, 255));
  pixels.show(); delay(1000); // Cian

  pixels.setPixelColor(0, pixels.Color(255, 0, 255));
  pixels.show(); delay(1000); // Magenta

  pixels.setPixelColor(0, pixels.Color(255, 255, 0));
```

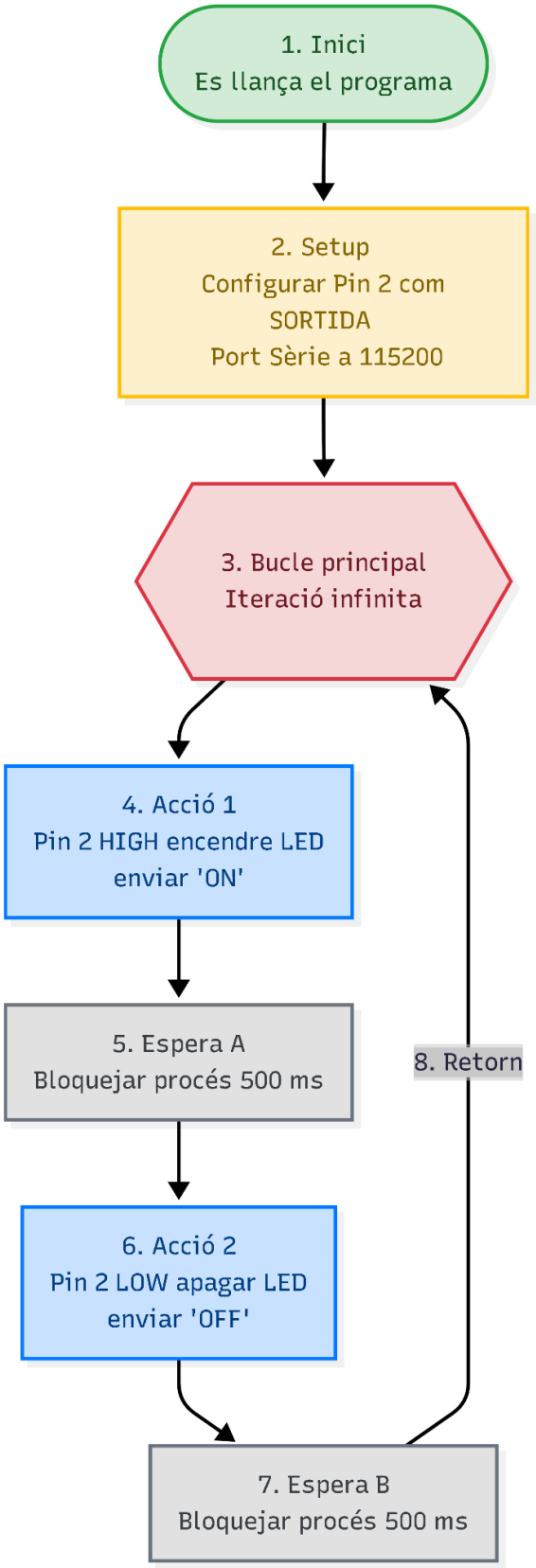
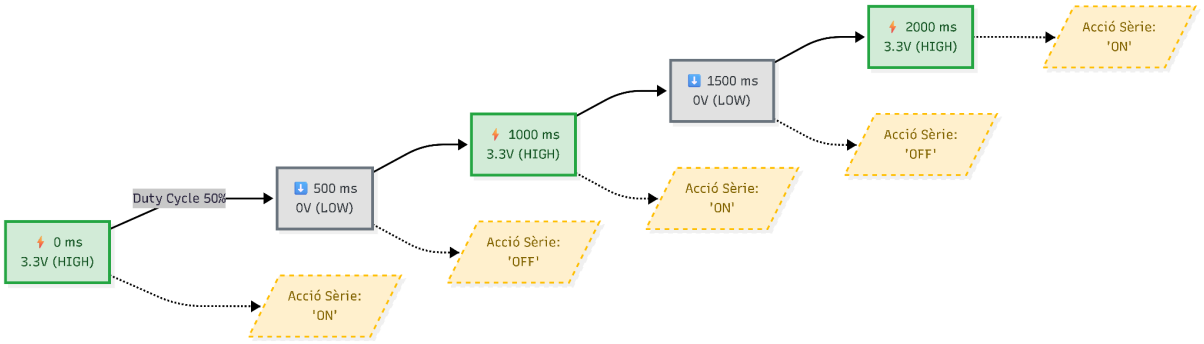
```
  pixels.show(); delay(1000); // Amarillo

  // Blanc i Apagat
  pixels.setPixelColor(0, pixels.Color(255, 255, 255));
  pixels.show(); delay(1000); // Blanco

  pixels.clear();
  pixels.show(); delay(100);
}
```

Explicació del codi:

- A setup() instanciem el LED i establim la brillantor molt baixa a només 10 tal com requereix la pràctica.
- Mitjançant funcions de diagnòstic internes (ESP.getChipModel, ESP.getEfuseMac), el microcontrolador llegeix la seva informació de maquinari i adreça MAC de fàbrica, escrivint-la pel port sèrie.
- Al bucle loop(), es crida setPixelColor() barrejant intensitats RGB del 0 al 255 per conformar els 6 colors bàsics més el blanc.
- L'ordre clau és pixels.show(), que envia físicament la informació de color des de la memòria cap al díode LED. Es posa un delay(1000) per mantenir el color 1 segon cadascun abans de netejar (clear).

5. Diagrames de Flux i de Temps

5.1 Diagrama de Flux (Blink + Serial Debug)



5.2 Diagrama de Temps (Blink 50% Duty Cycle)